TRAVELING SALESPERSON AND 2-OPT SWAP

Brady Farley
Calvin Pugmire
Rylee Albrecht
Heather Schone

ABSTRACT

The famous traveling salesperson problem (TSP) continues to be an area of research due to its NP-Complete nature. Various algorithms exist to attempt to find good solutions quickly. One such type of algorithm, called 2-opt swap is a local search algorithm that attempts to improve upon a previously found solution. Our team implemented a version of 2-opt swap and discovered that it is very fast and can determine a solution for a large number of cities. There are drawbacks to 2-opt as well. Overall, 2-opt performs rather quickly for large city sizes and provides a solution slightly better than a greedy approach.

INTRODUCTION

The 2-opt swap approach to the traveling salesperson problem (TSP) improves on a greedy approach by using a local search mechanism. Initially described by G.A. Croes in 1958, the algorithm works by performing a series of inversions on a trial solution to obtain a solution closer to optimal.[1] The idea for 2-opt initially sprouted while researching various solutions to TSP and discovering an article by Pedram Ataee in which he compared different approaches and determined that 2-opt was the best of the three he chose.[2] After discovering Ataee's article, we further researched 2-opt and ultimately decided on this approach because we wanted to examine how a local search implementation would fare when compared to other algorithms in this famous TSP problem. Given a route of cities A={a, b, c, d, e}, 2-opt swaps two of the cities in the route; for example: B={a, c, b, d, e}, if the cost of route B is less than the cost of route A, route B becomes the new best solution found and route B is swapped until a new best solution is found. This process continues until no new improvements are possible and the final best solution is returned. The initial route of cities is found through a greedy approach. Later in this report, we will discuss the benefits and disadvantages of this approach, as well as empirical results when compared to other algorithms.

ALGORITHM EXPLANATION

As has already been explained, two-opt swaps the cities in an attempt to find a better route. Many forms of 2-opt operate by swapping cities in the route to avoid edges crossing.[3] Although this was the main idea when we chose this specific algorithm, we implemented a version to just test any inversion, regardless of whether edges cross or not. Our team's implementation also adds a speed element to this approach. For city sizes less than 80 (this was an arbitrary number we picked to suit our purposes), we implement a generic form of 2-opt. For example if the initial tour is: {a, b, c, d}, then the first iteration would check tours: {b, a, c, d}, {c, b, a, d}, {d, b, c, a} and choose the best and perform the next iteration on the best until no improvement is made. Additionally, we implemented a faster approach for city sizes larger than 80, which does not give as optimal of solutions but is able to handle the larger city size. This faster

[1] Croes, G. A. (1958). A Method for Solving Traveling-Salesman Problems. *Operations Research, 6*(6), 791-812. http://www.jstor.org.byu.idm.oclc.org/stable/167074

[2] Ataee, P. (2020, June 14,). How to Solve the Traveling Salesman Problem — A Comparative Analysis. https://towardsdatascience.com/how-to-solve-the-traveling-salesman-problem-a-comparative-analysis-39056a916c9f

[3] Weru, L.*11 Animated Algorithms for the Traveling Salesman Problem.* Stem Lounge. Retrieved November 30, 2022, from https://stemlounge.com/animated-algorithms-for-the-traveling-salesman-problem/

approach works by using the tour and moving on to the next iteration whenever *any* improvement is found. In the example above, if {b, a, c, d} is an improvement over the initial tour, then the second iteration will start immediately on this better tour. There are several observable advantages and disadvantages to this approach.

Advantages

1. *Speed*
   This algorithm works quickly, while still providing a "good-enough" solution. Each iteration of the regular approach is approximately $O(n^2)$, where n is the number of cities. It performs multiple iterations, so the overall time complexity is slightly worse than $O(n^2)$, but it is still fast. The complete time complexity will be discussed later on.
2. *Provides solution in given time constraint*
   As long as an initial solution is procurable, this approach works with time constraints. Ideally, it finishes before the time runs out, but if not, it can still return a good solution of what it found so far.
3. *Handles large city sizes*
   Probably the best advantage it has over other approaches to TSP is that it can handle city sizes into the hundreds, relatively well.

Disadvantages

1. *Relies on an initial solution*
   Unfortunately, 2-opt swap does not develop its own initial solution. In our team's implementation, the initial solution is generated from a greedy approach. This means that the ultimate run time and problem set size can never do better than greedy.
2. *Traps in local optima*
   Like other local search algorithms, 2-opt can find a good local optimum but will get trapped searching for that optimum. It will only be able to obtain the absolute best solution if the rare case occurs that the local optima in the search area is also the global optima; unfortunately, this often isn't the case.

GREEDY ALGORITHM

Our implementation of the greedy algorithm is based on our class discussion that the greedy approach is focused on each next decision and chooses based on which gives the most immediate benefit. For TSP we implemented this idea by starting at the first city, checking the cost to each other city, and taking the cheapest path to the next city. We continued taking the cheapest path from among those not visited yet until we either visited all of the cities and could return back to the start city, or we reached a point that would make the cost infinity. If a solution is not found the start city is moved to the next city and runs again. This is done until a solution is found or all cities have been the start city. The solution is then returned if one was found, otherwise if no solution is possible, it returns infinity.

GREEDY COMPLEXITY

The greedy algorithm has a while loop that continues until a solution is found, which has a time complexity best case O(1) and worst case O(n). Within this while loop is where each city has to check its cost to each other city which adds two more nested for

loops of time complexity O(n). Then within the last for loop it checks within the route so far to make sure the current city is not already in the route before checking the cost to it. This check has a time complexity of O(log n) since the size of the list it searches through starts at size 0 and incrementally increases to size n. After this is when we append to the route which ends up being a list of n length that adds a space complexity of O(n). Overall since all of these time complexities are nested, we end up with a time complexity of n*n*n*logn = O(n^3log n) and a space complexity of O(n).

ALGORITHM COMPLEXITY

n = number of cities. m = number of times improved.		
	Time:	Space:
fancy()	reg_fancy or fast_fancy = O(2^n)	Route lists = O(n)
reg_fancy()	greedy + m(n(logn(swapVals+TSPSol))) = O(2^n)	Route lists = O(n)
fast_fancy()	greedy + m(n(logn)+swapVals) = O(2^n)	Route lists = O(n)
greedy()	n(n(n(logn))) = O((n^3)log(n))	Route lists = O(n)
swapVals()	deepcopy() = O(n)	Route lists = O(n)
TSPSol()	_costOfRoute() = O(n)	Route list = O(n)

THEORETICAL VS. EMPIRICAL

City #	Empirical (not including greedy %):	Line of best fit for empirical:
15	Normal fancy: 0.00116	0.02*2^(0.2*(n-29.3)): 0.0028
30	Normal fancy: 0.0192	0.02*2^(0.2*(n-29.3)): 0.022
60	Normal fancy: 1.411	0.02*2^(0.2*(n-29.3)): 1.411
100	Fast fancy: 0.0322	0.00000655(n^2)logn: 0.131
200	Fast fancy: 0.1592	0.00000655(n^2)logn: 0.603
500	Fast fancy: 4.2752	0.00000655(n^2)logn: 4.42
1000	Fast fancy: 19.648	0.00000655(n^2)logn: 19.65

The empirical normal fancy section of our algorithm matches our theoretical time complexity of O(2^n), albeit with some changes in constants. The empirical fast fancy section of our algorithm is faster than the theoretical, with a time complexity in the O((n^2)logn) range. This is explained by 2-opt's tendency to run faster than the worst case of O(2^n), especially when alterations have been made to make it faster (as was the case with the fast fancy section).

DATA ANALYSIS

For our data, we ran the four different approaches of Random, Greedy, Branch and Bound, and our own 2-Opt swapping algorithm. The data was collected for differing numbers of cities with 15 being the smallest and 10,000 being the largest. The largest size was determined based on how large the largest algorithm went before it timed out at 10 minutes (for our case, Greedy timed out at 10,000 cities).

The Random approach was only able to handle up to 60 cities. After that, it would hit the 10 minute limit. The Branch and Bound approach timed right after the initial 15 city test (although, it did find the best solution for 15 cities between the four approaches). Greedy was able to handle the highest number of cities, capping out at 10,000, This is because it was just looking for the first (lowest) route that formed a cycle. As for our 2-Opt approach, it finally capped out at 2,500 cities, which means it was still able to handle over half of the tests.

When it came to determining path lengths, Random was clearly the worst. This is because it simply keeps iterating through random permutations of cities until it finds a cycle. Greedy was a definite improvement

from the Random approach. Not only was it able to handle much larger amounts of cities, the path lengths compared to Random were significantly better. On average, Greedy resulted in path lengths that were 52% better. As mentioned earlier, Branch and Bound resulted in the lowest path length for 15 cities among the four approaches. It had a 21% improvement on Greedy's 38% improvement of Random. Therefore, Branch and Bound is able to find the best solutions, but it takes significantly longer than the other approaches. Our own 2-Opt approach acted as a kind of middle ground between Greedy and Branch and Bound. It consistently resulted in smaller path lengths than Greedy, like Branch and Bound, and it was able to finish the majority of tests within the 10 minute time limit, like Greedy. The 2-Opt algorithm, on average, was 5.7% better than Greedy when it came to path lengths.

When it comes to choosing what algorithm to use, it depends on your data and what you care more about. If you have a very large amount of cities and just need a decent path length, then Greedy would be best. If you want the shortest path possible and have a lot of time to calculate it, Branch and Bound would be best. If you have a decent number of cities and want a better solution than Greedy, 2-Opt would be best.

FUTURE PLANS

Since 2-opt is a local search algorithm, our main focus if we had more time would be to find a good way of escaping local optima traps. As our algorithm stands, it continues to iterate through the loop until there is no better cost from all possible swaps of a path. This finds the local optimum but may not be the overall optimum. There are a few ways that we would go about solving this problem. One way would be to run our algorithm with several different starting paths and take the smallest cost from among them. Another way would be to add some cases where the algorithm takes a path that is larger than the current best path when there is no improvement, in an attempt to escape the local optima.

	Random		Greedy		Branch and Bound			2-Opt		
# Cities	Time (sec)	Path Length	Path Length	% of Random	Time (sec)	Path Length	% of Greedy	Time (sec)	Path Length	% of Greedy
15	0.0011	18903	11773	0.62	0.19	9299	0.79	0.029	11320	0.96
30	0.049	38409	17352	0.45	TB	TB	TB	0.96	17064	0.98
60	49.30	79863	29124	0.36	TB	TB	TB	14.11	26265	0.90
100	TB	TB	39017	TB	TB	TB	TB	0.46	36265	0.93
200	TB	TB	57173	TB	TB	TB	TB	3.98	54744	0.96
500	TB	TB	106341	TB	TB	TB	TB	53.44	97646	0.92
1000	TB	TB	162048	TB	TB	TB	TB	392.96	153328	0.95
2500	TB	TB	289314	TB	TB	TB	TB	TB	TB	TB
5000	TB	TB	450080	TB	TB	TB	TB	TB	TB	TB
10000	TB	TB	TB	TB	TB	TB	TB	TB	TB	TB

EXAMPLE RESULTS

To further illustrate several of the differences between the greedy approach and the 2-opt approach, below are some example runs.

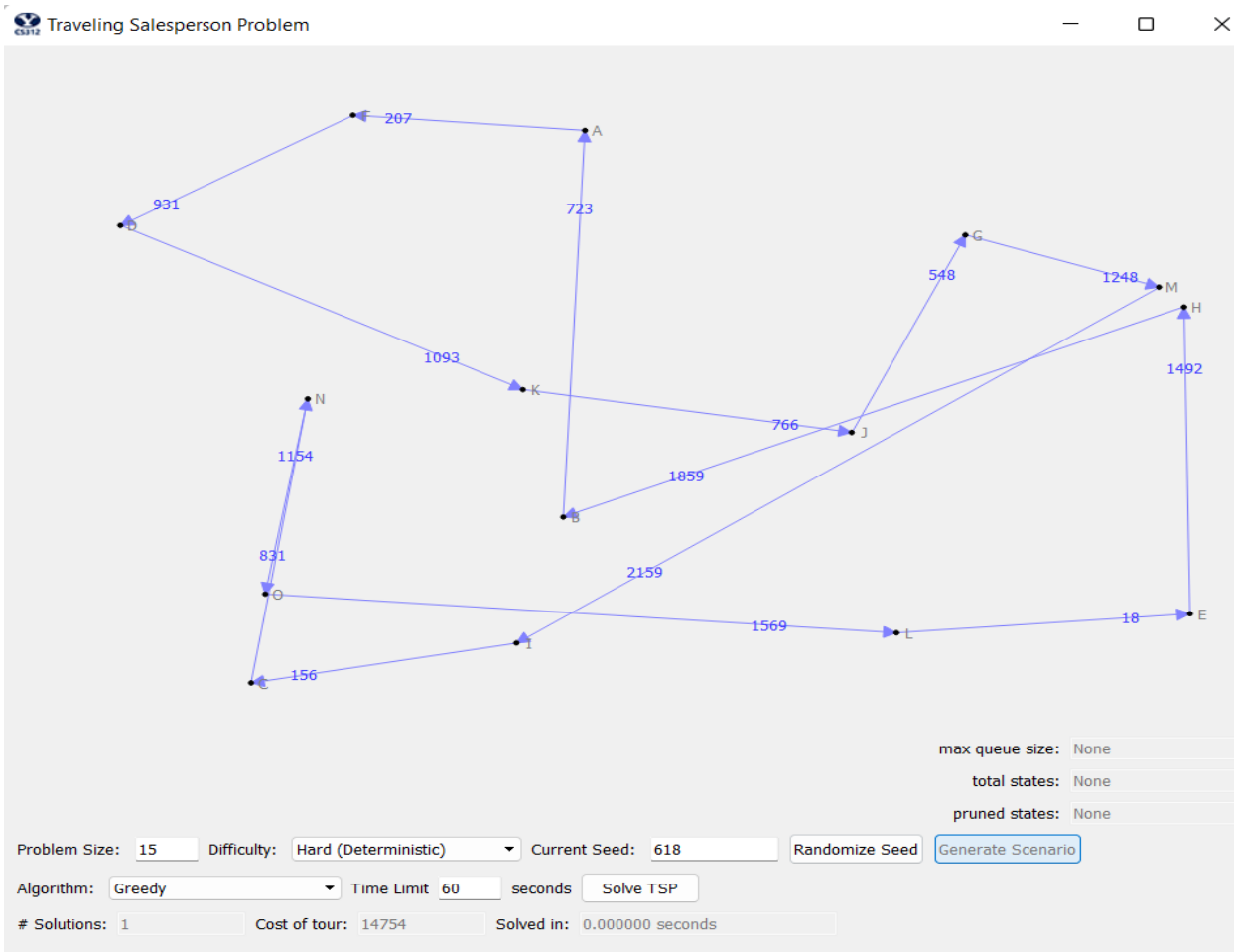

Figure 2: Example run using the greedy approach on a problem with 15 cities and a total route of 14754.

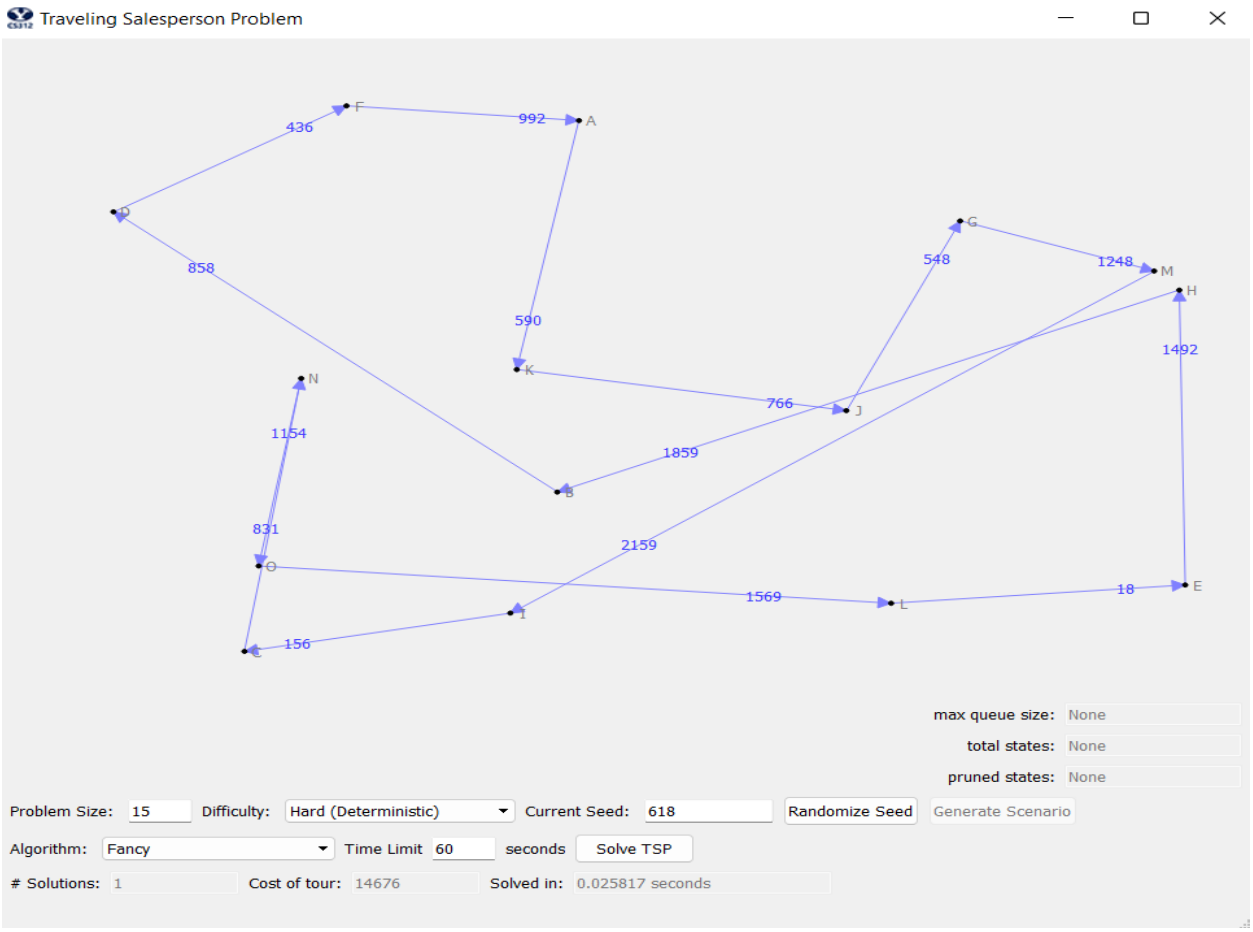

Figure 3: Example run using the 2-opt approach on a problem with 15 cities and a total route of 14676, a 1% improvement over greedy.

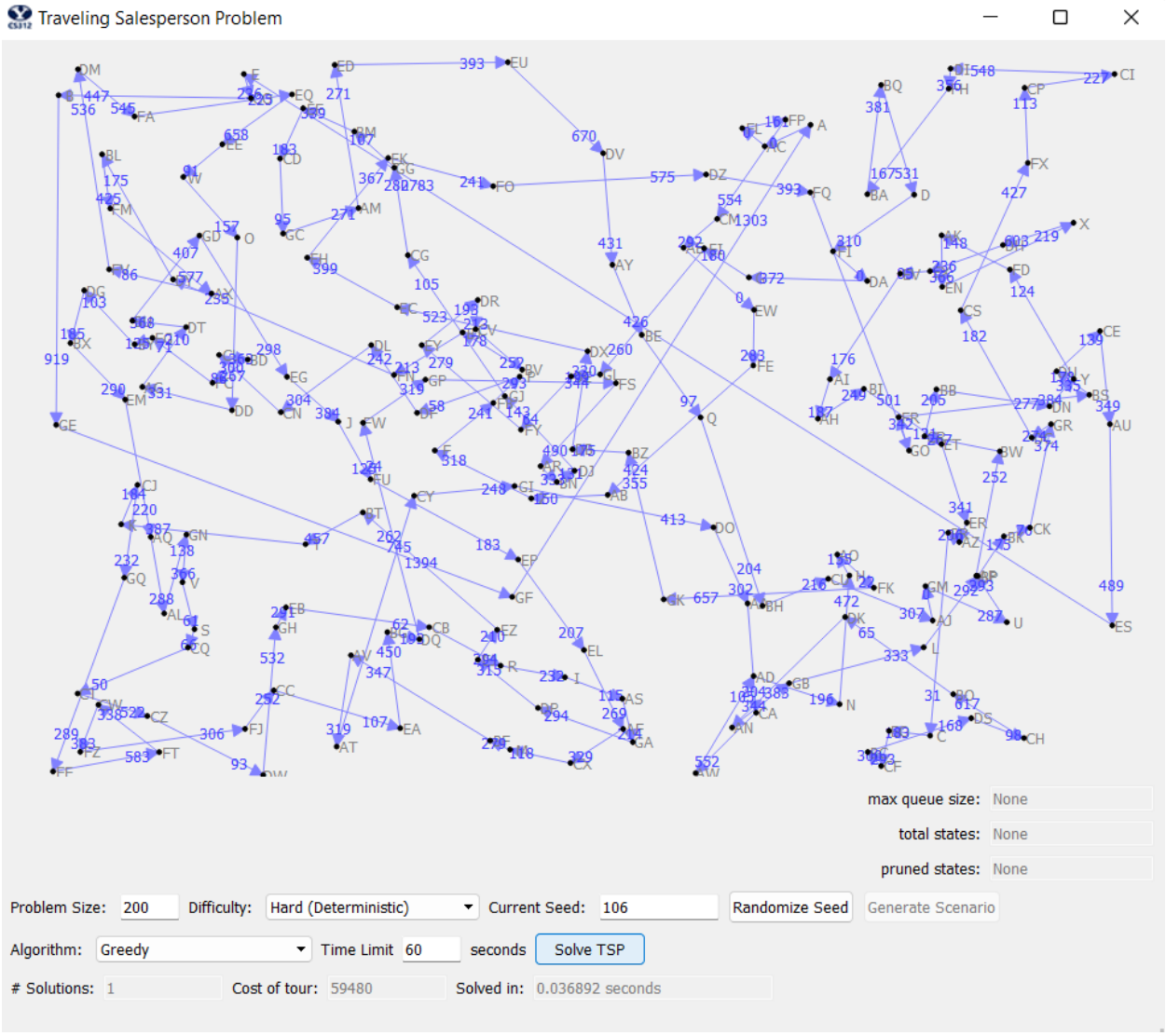

Figure 4: Example run using the greedy approach on a problem with 200 cities and a total route of 59480.

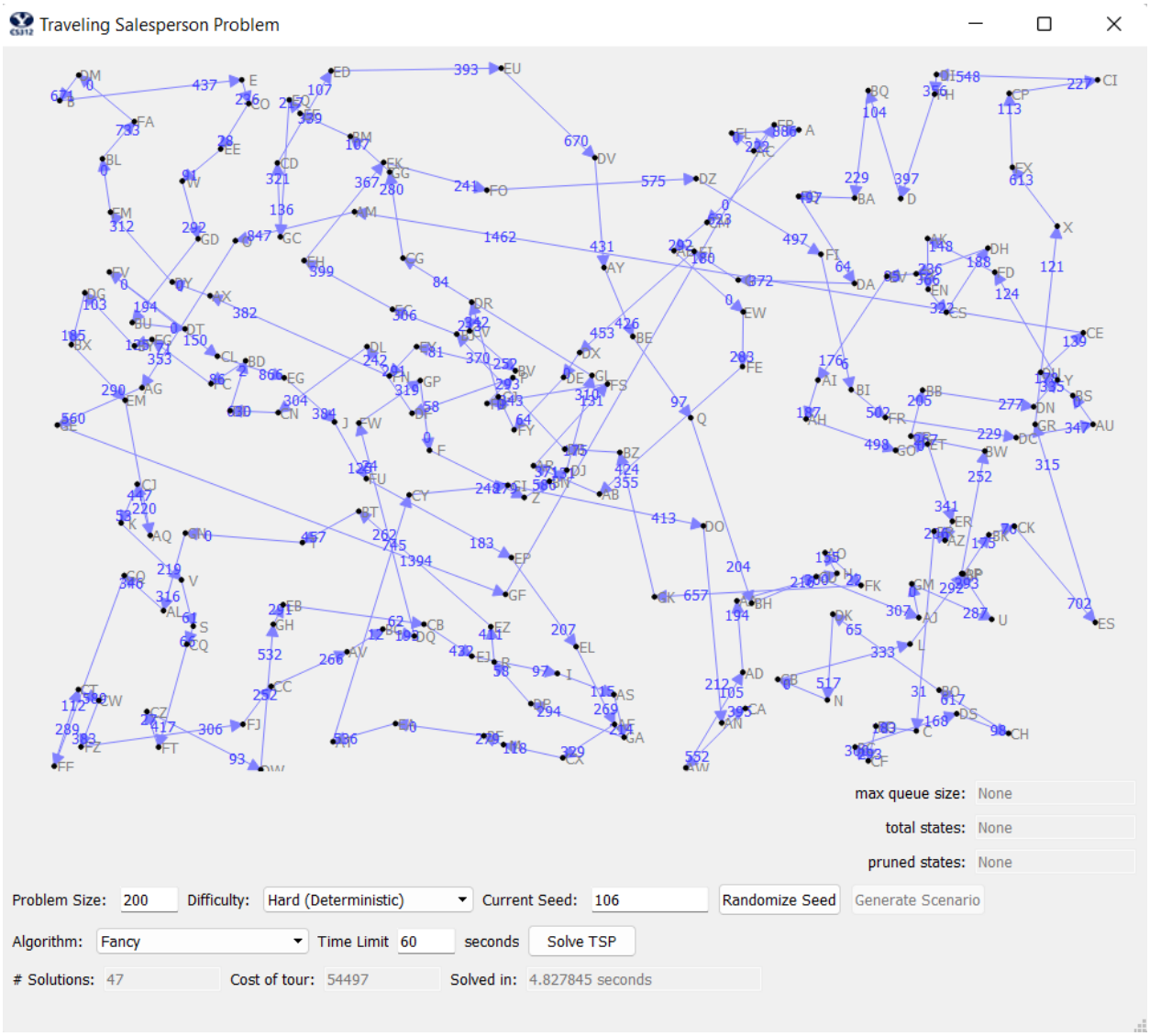

Figure 5: Example run using the 2-opt approach on a problem with 200 cities and a total route of 54497, with an 8% improvement over greedy.